SECURITY ASSESSMENT

Accretion

Provided by Accretion Labs Pte Ltd. for Solana
February 10, 2026
A25SFR3

Solana

AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Auditor	Niklas Breitfeld (niklas@accretion.xyz)
Auditor	Mahdi Rostami (mahdi@accretion.xyz)

CLIENT

Solana (https://solana.org) engaged Accretion to conduct a security assessment of Solana Record Service, a public good program that allows you to create, update, and delete records for names and TLDs

ENGAGEMENT TIMELINE

09 Oct — Project Kickoff
Initial planning and scope definition

09 Oct — Assessment Begins
Security review and testing phase

16 Jan — Review Fixes
Security recommendations are given and implemented

10 Feb — Project Completion
Report delivery and on-chain confirmation

AUDITED CODE

Program 1

ProgramID: srsUi2TVUUCyGcZdopxJauk8ZBzgAaHHZCVUhm5ifPa

Repository: https://github.com/solana-foundation/solana-record-service

ASSESSMENT

The security assessment of solana's solana-record-service revealed a total of 15 findings across various severity levels: 1 High, 3 Medium, 4 Low, and 7 Informational issues, with no Critical vulnerabilities identified. The development team demonstrated strong remediation practices, successfully fixing all security-relevant issues during the audit period.

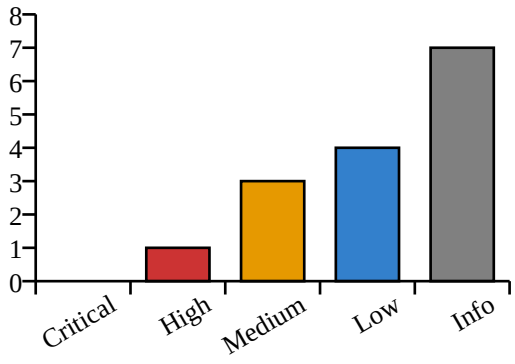
CODE ASSESSMENT

The codebase demonstrates solid architectural foundations with proper use of Solana program development patterns based on the Pinocchio framework. However, the audit identified several code quality concerns including scattered discriminator definitions, non-specific error handling, and suboptimal compute unit usage due to repeated PDA bump calculations.

KEY FINDINGS

The most critical findings included missing PDA validation in the create_record function, allowing unauthorized signers to create records. Additionally, the audit revealed authorization bypass issues where class authorities could modify any record data regardless of ownership, and problems with tokenized record deletion when NFTs are burned.

SEVERITY DISTRIBUTION



© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Solana Record Service

Link: https://github.com/solana-foundation/solana-record-service
Commit: ba0f64964eccc0b559ee01012d0d547aa52cec00
Program ID: srsUi2TVUUCyGcZdopxJauk8ZBzgAaHHZCVUhm5ifPa
Audit Result:

- **Audited Commit:** 80600132a833c47ea606a5f4b7d0975ad72c0664
- **Status:** unverified
- **Comment:** unverified

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-H1	Missing PDA check `create_record`	HIGH	FIXED
ACC-M1	Incorrect lamports calculation in `create_record`	MEDIUM	FIXED
ACC-M2	Class Authority Can Modify Any Record Data Regardless of Ownership	MEDIUM	FIXED
ACC-M3	Anyone Can Delete Tokenized Records when Tokenized Record NFT gets burned	MEDIUM	FIXED
ACC-L1	Can't Burn Frozen Tokens	LOW	FIXED
ACC-L2	Incorrect validation of Token22 Mint/Token accounts	LOW	FIXED
ACC-L3	Frozen State Not Enforced for UpdateRecordData	LOW	FIXED
ACC-L4	Missing Frozen State Check in UpdateClassMetadata and UpdateClassAuthority	LOW	FIXED
ACC-I1	`MAX_CLASS_NAME_LEN` vs PDA seed-length mismatch	INFO	FIXED
ACC-I2	check_owner_or_delegate doesn't check owner type at the beginning	INFO	FIXED
ACC-I3	Unsafe bool deserialization in freeze_record	INFO	FIXED
ACC-I4	No Bump Storage Causes Wasteful Compute Unit Usage	INFO	WONTFIX
ACC-I5	Non-Specific Error Codes Hinder Debugging	INFO	WONTFIX
ACC-I6	Arbitrary Debug Logging Statements Waste Compute Units	INFO	FIXED
ACC-I7	Discriminators Not Defined in Centralized Location	INFO	WONTFIX

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

DETAILED ISSUES

ACC-H1 Missing PDA check `create_record` HIGH FIXED

Description

We have found that the `create_record` does not enforce the record PDA and allows any signer account to be used.

PoC to added in `program/src/tests.rs`

```
#[test]
fn test_non_canonical_record_creation() {
    // Setup: Create a permissionless class
    let (class, class_data) = keyed_account_for_class_default();
    let (owner, owner_data) = keyed_account_for_owner();
    let (system_program, system_program_data) = keyed_account_for_system_program();

    // Use an arbitrary keypair instead of the PDA
    let arbitrary_record_key = Pubkey::new_from_array([0xef; 32]);

    let seed = b"test";

    let (correct_pda, _bump) = Pubkey::find_program_address(
        &[b"record", class.as_ref(), seed],
        &SOLANA_RECORD_SERVICE_ID,
    );

    assert_ne!(arbitrary_record_key, correct_pda,);

    use solana_program::instruction::{AccountMeta, Instruction};

    let accounts = vec![
        AccountMeta::new_readonly(owner, true),
        AccountMeta::new(owner, true),
        AccountMeta::new(class, false),
        AccountMeta::new(arbitrary_record_key, true), // <- BUG
        AccountMeta::new_readonly(system_program, false),
        AccountMeta::new_readonly(SOLANA_RECORD_SERVICE_ID, false),
    ];

    let expiration: i64 = 0;
    let data = b"test";

    let mut instruction_data = vec![4u8];
    instruction_data.extend_from_slice(&expiration.to_le_bytes());
    instruction_data.push(seed.len() as u8);
    instruction_data.extend_from_slice(seed);
    instruction_data.extend_from_slice(data);

    let instruction = Instruction {
        program_id: SOLANA_RECORD_SERVICE_ID,
        accounts,
        data: instruction_data,
    };

    let expected_record_data = Record {
        discriminator: 2,
        class,
        owner_type: 0,
        owner,
        is_frozen: false,
        expiry: 0,
        seed: make_u8prefix_vec_u8(seed),
        data: RemainderVec::<u8>::try_from_slice(data).unwrap(),
    };
    let mollusk = Mollusk::new(
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
        &SOLANA_RECORD_SERVICE_ID,
        "../target/deploy/solana_record_service",
    );

    mollusk.process_and_validate_instruction(
        &instruction,
        &[
            (owner, owner_data),
            (class, class_data),
            (arbitrary_record_key, Account::default()), // arbitrary account instead of PDA
            (system_program, system_program_data),
        ],
        &[
            Check::success(),
            Check::account(&arbitrary_record_key)
                .data(&expected_record_data.try_to_vec().unwrap())
                .build(),
        ],
    );
}
```

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/create_record.rs#L142

Relevant Code

```
let bump: [u8; 1] = [try_find_program_address(&seeds, &crate::ID)
            .ok_or(ProgramError::InvalidArgument)?
            .1];
```

Mitigation Suggestion

Check the key

Remediation

Fixed in https://github.com/solana-foundation/solana-record-service/commit/a6d21f005305ce233a3592b94255f379de29245f.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M1 Incorrect lamports calculation in `create_record`

MEDIUM FIXED

Description

We have found that the lamports calculation is incorrect when accounts are toped up in `create_record`.

For instance consider the following:

- `rent = 10`
- `self.accounts.record.lamports() = 1`
- `lamports = 10 - 1 = 9`
- `self.accounts.record.lamports() > 0` is `true`
- `self.accounts.record.lamports() < lamports` is true
- `lamports - self.accounts.record.lamports = 9 - 1 = 8` is transferred to `ctx.accounts.record`
- `ctx.accounts.record` now has `1 + 8 = 9` lamports instead of `10`

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/create_record.rs#L173

Relevant Code

```
if self.accounts.record.lamports() < lamports {
                Transfer {
                    from: self.accounts.payer,
                    to: self.accounts.record,
                    lamports: lamports - self.accounts.record.lamports(),
                }
                .invoke()?;
            }
```

Mitigation Suggestion

`self.accounts.record.lamports()` should be compared to `rent` https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/create_record.rs#L169 and `lamports` i.e delta between expected and current should be transferred https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/create_record.rs#L173

Remediation

Fixed in https://github.com/solana-foundation/solana-record-service/commit/127dd9fa34855bc35580ae0bc9e80b233a3f45d5.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M2 Class Authority Can Modify Any Record Data Regardless of Ownership

MEDIUM FIXED

Description

We found that the `UpdateRecordData` and `UpdateRecordExpiry` instructions only check if the signer is the class authority, not if they are the record owner. This means that in permissionless classes (where anyone can create records), the class authority has unrestricted access to modify any record's data and expiry, even records they don't own.

This creates a severe trust issue where:

• Users can create records in permissionless classes

• The class authority can arbitrarily modify or censor any user's record data

• Record owners have no way to protect their data from class authority interference

• Even freezing the record doesn't help, as the class authority can update it (frozen check is missing at the instruction level for UpdateRecordData)

This is particularly problematic because:

• Record creators may assume they control their own records

• The class being "permissionless" implies users have autonomy

• There's no way for record owners to prevent class authority tampering

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/update_record.rs#L31-L56

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/update_record.rs#L95-L133

Relevant Code

```
// update_record.rs lines 31-56
impl<'info> TryFrom<&'info [AccountInfo]> for UpdateRecordAccounts<'info> {
    type Error = ProgramError;

    fn try_from(accounts: &'info [AccountInfo]) -> Result<Self, Self::Error> {
        let [authority, payer, record, class, _system_program] = accounts else {
            return Err(ProgramError::NotEnoughAccountKeys);
        };

        if !authority.is_signer() {
            return Err(ProgramError::MissingRequiredSignature);
        }

        // ISSUE: Only checks if authority is the class authority
        // Does NOT check if authority is the record owner
        // Check if authority is the record owner or has a delegate
        Class::check_authority(class, authority)?;

        // Check if the Record is correct
        Record::check_program_id_and_discriminator(record)?;

        // Check if the class is the correct class
        if class.key().ne(&record.try_borrow_data()?[CLASS_OFFSET..CLASS_OFFSET + size_of::<Pubkey>()]) {
            return Err(ProgramError::InvalidAccountData);
        }

        Ok(Self { payer, record })
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    }
}
```

Mitigation Suggestion

Given the comment saying "check if authority is the record owner or has a delegate" the call in line 44 should be

```
Record::check_owner_or_delegate_or_deleted
```

Remediation

Fixed in https://github.com/solana-foundation/solana-record-service/commit/6f9da94b5797fb2aec9be6bff543bc751ac64fa5.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M3 Anyone Can Delete Tokenized Records when Tokenized Record NFT gets burned

MEDIUM FIXED

Description

We found that when a record is tokenized (owner type is Token), the check_owner_or_delegate_or_deleted function allows ANYONE to delete the record if the mint supply is zero, without requiring any authorization. The function returns early at line 146 after closing the mint, bypassing all further authority checks. The NFT record needs to be burned outside the record service program, as burning through the record program will automatically close the mint and update the owner type.

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/state/record.rs#L100-L169

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/delete_record.rs#L32-L47

Relevant Code

```
// record.rs lines 100-169 - check_owner_or_delegate_or_deleted
pub fn check_owner_or_delegate_or_deleted(
    record: &AccountInfo,
    class: Option<&AccountInfo>,
    authority: &AccountInfo,
    mint: Option<&AccountInfo>
) -> Result<(), ProgramError> {
    // Check the program id and the discriminator
    Self::check_program_id_and_discriminator(record)?;

    let data = record.try_borrow_data()?;

    // Check if the Mint has been burned without passing through the BurnTokenizedRecord instruction
    if data[OWNER_TYPE_OFFSET].eq(&(OwnerType::Token as u8)) {
        let mint = mint.ok_or(ProgramError::InvalidAccountData)?;

        if mint.key().ne(&data[OWNER_OFFSET..OWNER_OFFSET + size_of::<Pubkey>()]) {
            return Err(ProgramError::InvalidAccountData);
        }

        // VULNERABILITY: Only checks supply is 0, not who is calling!
        if Mint::get_supply(mint)? != 0 {
            return Err(ProgramError::InvalidAccountData);
        }

        // Close the Mint and get back the rent
        let bump = [
            try_find_program_address(&[b"mint", record.key()], &crate::ID)
                .ok_or(ProgramError::InvalidArgument)?
                .1,
        ];

        let seeds = [
            Seed::from(b"mint"),
            Seed::from(record.key()),
            Seed::from(&bump),
        ];

        let signers = [Signer::from(&seeds)];

        // Close the mint account
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
        CloseAccount {
            account: mint,
            destination: authority,  // Rent goes to arbitrary 'authority' parameter!
            authority: mint,
        }
        .invoke_signed(&signers)?;

        return Ok(());  // EARLY RETURN - bypasses all authority checks below!
    }

    // These checks are NEVER reached for tokenized records:

    // Check if the authority is signer
    if !authority.is_signer() {
        return Err(ProgramError::MissingRequiredSignature);
    }

    // Check if the authority is the owner
    if authority
        .key()
        .eq(&data[OWNER_OFFSET..OWNER_OFFSET + size_of::<Pubkey>()])
    {
        return Ok(());
    }

    // Validate the delegate
    let class = class.ok_or(ProgramError::InvalidAccountData)?;
    if class.key().ne(&data[CLASS_OFFSET..CLASS_OFFSET + size_of::<Pubkey>()]) {
        return Err(ProgramError::InvalidAccountData);
    }

    Self::validate_delegate(class, authority)
}
```

Mitigation Suggestion

We have multiple options:

• Only allow the class authority to clean up, if we want to retain the functionality of when the record holder burns his token outside the record service program.

• Disallow deleting the record until the record has been burned through the record service program (and thus removing the ownership type `OwnerType::Token`)

Remediation

Fixed in https://github.com/solana-foundation/solana-record-service/commit/319373e292720fe80eb9a30a63ab300f117def11

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L1 Can't Burn Frozen Tokens

LOW FIXED

Description

Fe found that burn_tokenized_record will fail when attempting to burn a frozen token account, even though it tries to unfreeze it.

The burn flow:

1. Reads `is_frozen` from the token account. 2. If frozen, thaws the token account (CPI) so burning is possible. 3. Burns and closes the mint. 4. Updates the record:

- `update_is_frozen_unchecked(record, is_frozen)` (makes the record frozen!)
- `update_owner_unchecked(record, record_owner)` (fails because now the record is frozen!)
- `update_owner_type_unchecked(record, Pubkey)`

However, `Record::update_owner_unchecked` rejects updates if `record.is_frozen == 1`.

Therefore, when `is_frozen` is true, the code sets the record frozen flag first, then attempts to update owner/type and fails.

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/burn_tokenized_record.rs#L143-L156

Relevant Code

```
/// burn_tokenized_record.rs L143-L156
        unsafe {
            Record::update_is_frozen_unchecked(
                &mut self.accounts.record.try_borrow_mut_data()?,
                is_frozen,
            )?;
            Record::update_owner_unchecked(
                &mut self.accounts.record.try_borrow_mut_data()?,
                &record_owner,
            )?;
            Record::update_owner_type_unchecked(
                &mut self.accounts.record.try_borrow_mut_data()?,
                OwnerType::Pubkey,
            )?;
        };
```

Mitigation Suggestion

Write the frozen flag to the record last.

Remediation

Fixed in https://github.com/solana-foundation/solana-record-service/commit/f9c0755de60c6c6f0b33fc25efef02d13ef55c52

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L2 Incorrect validation of Token22 Mint/Token accounts

LOW FIXED

Description

We found that the functions used to validate that an account is a Token22 Mint or Token account do not correctly account for Multisig accounts. The functions work by checking the discriminator at offset 165, which only exists when extensions are present. However, a Multisig account belonging to the Token 22 program may have an arbitrary byte at this offset, allowing it to pass as either a token account or a mint.

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/token2022/state.rs#L25-L47 https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/token2022/state.rs#L88-L96

Relevant Code

```
/// state.rs L25-L47
/// Mint
    /// # Safety
    /// Token Program ID is not checked
    pub unsafe fn check_discriminator_unchecked(data: &[u8]) -> Result<(), ProgramError> {
        if data[TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET].ne(&MINT_DISCRIMINATOR) {
            return Err(ProgramError::InvalidAccountData);
        }

        Ok(())
    }

    pub fn check_discriminator(account_info: &AccountInfo) -> Result<bool, ProgramError> {
        if unsafe { account_info.owner().ne(&TOKEN_2022_PROGRAM_ID) } {
            return Ok(false);
        }

        let data = account_info.try_borrow_data()?;

        if data[TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET].ne(&MINT_DISCRIMINATOR) {
            return Ok(false);
        }

        Ok(true)
    }

/// state.rs L88-L96
/// Token Account
    /// # Safety
    /// Token Program ID is not checked
    pub unsafe fn check_discriminator_unchecked(data: &[u8]) -> Result<(), ProgramError> {
        if data[TOKEN_2022_ACCOUNT_DISCRIMINATOR_OFFSET].ne(&TOKEN_ACCOUNT_DISCRIMINATOR) {
            return Err(ProgramError::InvalidAccountData);
        }

        Ok(())
    }
```

Mitigation Suggestion

Add a special case that checks if an account is a Multisig account, which is the case when it has a length of 355 bytes.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Remediation

Fixed in
https://github.com/solana-foundation/solana-record-service/commit/ee8903249e58e666a747124d560ed53613fdd8a7

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L3 Frozen State Not Enforced for UpdateRecordData

LOW FIXED

Description

We found that the `UpdateRecordData` instruction does not check if a record is frozen before allowing data modifications. While `update_owner_unchecked` and `update_expiry_unchecked` both check the frozen state and return an error, `update_data_unchecked` has no frozen check at all.

This creates an inconsistency where:

- `UpdateRecordExpiry` respects frozen state (via `update_expiry_unchecked` line 343)
- `TransferRecord` respects frozen state (via `update_owner_unchecked` line 319)
- `UpdateRecordData` IGNORES frozen state - no check in `update_data_unchecked`

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/state/record.rs#L353-L387

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/update_record.rs#L80-L92

Relevant Code

```
// record.rs lines 357-387 - update_data_unchecked has NO frozen check
pub unsafe fn update_data_unchecked(
    record: &'info AccountInfo,
    payer: &'info AccountInfo,
    data: &'info str,
) -> Result<(), ProgramError> {
    let seed_len = {
        let data_ref = record.try_borrow_data()?;
        data_ref[SEED_LEN_OFFSET] as usize
    };

    let offset = seed_len + SEED_LEN_OFFSET + size_of::<u8>();
    let current_len = record.data_len();
    let new_len = offset + data.len();

    if new_len != current_len {
        resize_account(record, payer, new_len, new_len < current_len)?;
    }

    {
        let mut data_ref = record.try_borrow_mut_data()?;
        if data_ref[DISCRIMINATOR_OFFSET].ne(&Self::DISCRIMINATOR) {
            return Err(ProgramError::InvalidAccountData);
        }
        // MISSING: No check for IS_FROZEN_OFFSET here!
        let data_buffer = unsafe {
            core::slice::from_raw_parts_mut(data_ref.as_mut_ptr().add(offset), data.len())
        };
        data_buffer.clone_from_slice(data.as_bytes());
    }

    Ok(())
}

// Compare with update_owner_unchecked (lines 314-332) - HAS frozen check
pub unsafe fn update_owner_unchecked(
    data: &mut RefMut<'info, [u8]>,
    new_owner: &Pubkey,
) -> Result<(), ProgramError> {
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    // Check if the record is frozen
    if data[IS_FROZEN_OFFSET].eq(&1u8) {
        return Err(ProgramError::InvalidAccountData);
    }
    // ...
}

// Compare with update_expiry_unchecked (lines 338-351) - HAS frozen check
pub unsafe fn update_expiry_unchecked(
    data: &mut RefMut<'info, [u8]>,
    new_expiry: i64,
) -> Result<(), ProgramError> {
    // Check if the record is frozen
    if data[IS_FROZEN_OFFSET].eq(&1u8) {
        return Err(ProgramError::InvalidAccountData);
    }
    // ...
}
```

Mitigation Suggestion

Add frozen state check to `update_data_unchecked` to match the pattern used in other update functions:

```
pub unsafe fn update_data_unchecked(
    record: &'info AccountInfo,
    payer: &'info AccountInfo,
    data: &'info str,
) -> Result<(), ProgramError> {
    // ADD: Check if the record is frozen FIRST
    {
        let data_ref = record.try_borrow_data()?;
        if data_ref[IS_FROZEN_OFFSET].eq(&1u8) {
            return Err(ProgramError::InvalidAccountData);
        }
    }

    let seed_len = {
        let data_ref = record.try_borrow_data()?;
        data_ref[SEED_LEN_OFFSET] as usize
    };

    let offset = seed_len + SEED_LEN_OFFSET + size_of::<u8>();
    let current_len = record.data_len();
    let new_len = offset + data.len();

    if new_len != current_len {
        resize_account(record, payer, new_len, new_len < current_len)?;
    }

    {
        let mut data_ref = record.try_borrow_mut_data()?;
        if data_ref[DISCRIMINATOR_OFFSET].ne(&Self::DISCRIMINATOR) {
            return Err(ProgramError::InvalidAccountData);
        }
        let data_buffer = unsafe {
            core::slice::from_raw_parts_mut(data_ref.as_mut_ptr().add(offset), data.len())
        };
        data_buffer.clone_from_slice(data.as_bytes());
    }

    Ok(())
}
```

Remediation

Fixed in https://github.com/solana-foundation/solana-record-service/commit/8deed64b69f3be0874d476eea12a50762342e7c1

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L4 Missing Frozen State Check in UpdateClassMetadata and UpdateClassAuthority

LOW FIXED

Description

We found that the `UpdateClassMetadata` and `UpdateClassAuthority` instructions do not verify whether the class is frozen before allowing updates. This allows class authority and metadata to be modified even after the class has been frozen, violating the intended immutability of frozen classes.

Impact:

- Violates the intended immutability of frozen classes
- Class authority can be changed on frozen classes, potentially allowing unauthorized control
- Metadata can be changed on frozen classes, violating data integrity expectations

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/update_class.rs#L28-L40

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/update_class.rs#L70-L84

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/update_class.rs#L112-L126

Relevant Code

```
// update_class.rs lines 28-40
impl<'info> TryFrom<&'info [AccountInfo]> for UpdateClassAccounts<'info> {
    type Error = ProgramError;

    fn try_from(accounts: &'info [AccountInfo]) -> Result<Self, Self::Error> {
        let [authority, payer, class, _system_program] = &accounts else {
            return Err(ProgramError::NotEnoughAccountKeys);
        };

        Class::check_authority(class, authority)?;
        // MISSING: No check for is_frozen state

        Ok(Self { payer, class })
    }
}
```

Mitigation Suggestion

Add frozen state check in `UpdateClassAccounts::try_from`:

```
Class::check_authority(class, authority)?;

// Add frozen check
let data = class.try_borrow_data()?;
if data[IS_FROZEN_OFFSET] == 1 {
    return Err(ProgramError::InvalidAccountData);
}
```

Remediation

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Fixed in
https://github.com/solana-foundation/solana-record-service/commit/b1b54cf5573f2f24b1ce2ae07c42837b91e0a315

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I1 `MAX_CLASS_NAME_LEN` vs PDA seed-length mismatch

INFO FIXED

Description

Solana PDA seed components are limited to 32 bytes, but `MAX_CLASS_NAME_LEN` suggests names up to 255 bytes.

In practice, any name > 32 bytes will fail PDA derivation.

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/state/class.rs#L27-L27 https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/create_class.rs#L141-L145

Relevant Code

```
/// class.rs L27-L27
    pub const MAX_CLASS_NAME_LEN: usize = 0xff;

/// create_class.rs L141-L145
        let seeds = [
            b"class",
            self.accounts.authority.key().as_ref(),
            self.name.as_bytes(),
        ];
```

Mitigation Suggestion

Make the limit 32 bytes to avoid confusing failures.

Remediation

Fixed in
https://github.com/solana-foundation/solana-record-service/commit/f78895d717811f1e9510ded64a8b0d5bd68b57d1

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I2 check_owner_or_delegate doesn't check owner type at the beginning

INFO FIXED

Description

We found that `check_owner_or_delegate` doesn't check owner type until after the authority key check has failed. This mean that we can use a mint key and sign the instruction, returning early, even when owner type is tokenized.

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/state/record.rs#L171-L198

Relevant Code

```
/// record.rs L171-L198
    #[inline(always)]
    pub fn check_owner_or_delegate(
        record: &AccountInfo,
        class: Option<&AccountInfo>,
        authority: &AccountInfo,
    ) -> Result<(), ProgramError> {
        // Check the program id and the discriminator
        Self::check_program_id_and_discriminator(record)?;

        // Check if the authority is signer
        if !authority.is_signer() {
            return Err(ProgramError::MissingRequiredSignature);
        }

        let data = record.try_borrow_data()?;

        // Check if the authority is the owner
        if authority
            .key()
            .eq(&data[OWNER_OFFSET..OWNER_OFFSET + size_of::<Pubkey>()])
        {
            return Ok(());
        }

        // Check if the owner type is pubkey
        if data[OWNER_TYPE_OFFSET].ne(&(OwnerType::Pubkey as u8)) {
            return Err(ProgramError::InvalidAccountData);
        }
```

Mitigation Suggestion

The owner type check should be at the beginning of the function.

Remediation

Fixed in
https://github.com/solana-foundation/solana-record-service/commit/e89d9f28070b2589e14bc5425211d44ec6a4620d

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I3 Unsafe bool deserialization in freeze_record

INFO FIXED

Description

In Rust, a bool must be 0x00 or 0x01; any other value is technically undefined behavior (https://doc.rust-lang.org/reference/types/boolean.html#r-type.bool.repr). Any `ByteReader::read_with_offset.` (e.g here) is technically UB.

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/freeze_record.rs#L72

Relevant Code

```
let is_frozen: bool = ByteReader::read_with_offset(ctx.data, IS_FROZEN_OFFSET)?;
```

Mitigation Suggestion

```
let v: u8 = ByteReader::read_with_offset(ctx.data, IS_FROZEN_OFFSET)?;
let is_frozen = match v { 0 => false, 1 => true, _ => return Err(ProgramError::InvalidInstructionData), };
```

Remediation

Fixed in https://github.com/solana-foundation/solana-record-service/commit/f08f340a9460ca5e764af4e8405336c3835d0622

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I4 No Bump Storage Causes Wasteful Compute Unit Usage

INFO WONTFIX

Description

The program recalculates PDA bumps using `try_find_program_address` on every instruction execution instead of storing the bump when the PDA is created. This wastes significant compute units as the bump search is performed via brute force (trying bump values from 255 down to 0 until a valid PDA is found).

This is most impactful with the Mint PDA as it will get calculated not only during the initial creation, but also in every operation: `delete_record.rs`, `freeze_tokenized_record.rs`, `burn_tokenized_record.rs`

Location

• https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/mint_tokenized_record.rs#L209-L215

• https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/state/record.rs#L134-L136

• https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/freeze_tokenized_record.rs#L115-L119

• https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/burn_tokenized_record.rs#L76-L80

Relevant Code

```
fn derive_mint_address_bump(&self) -> Result<[u8; 1], ProgramError> {
    let seeds = [b"mint", self.accounts.record.key().as_ref()];

    Ok([try_find_program_address(&seeds, &crate::ID)
        .ok_or(ProgramError::InvalidArgument)?
        .1])
}
// This is called EVERY time, even though bump never changes

// state/record.rs lines 134-136 - Recalculating in delete path
let bump = [
    try_find_program_address(&[b"mint", record.key()], &crate::ID)
        .ok_or(ProgramError::InvalidArgument)?
        .1,
];
```

Mitigation Suggestion

Store bump in account data

```
// For Class account, add bump field
pub struct Class<'info> {
    pub authority: Pubkey,
    pub is_permissioned: bool,
    pub is_frozen: bool,
    pub bump: u8,  // NEW: Store the bump
    pub name: &'info str,
    pub metadata: &'info str,
}

// During initialization, store the bump
let bump = try_find_program_address(&seeds, &crate::ID)
    .ok_or(ProgramError::InvalidArgument)?
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    .1;

// Write bump to account data
ByteWriter::write_with_offset(&mut data, BUMP_OFFSET, bump)?;

// On subsequent reads, just read the bump from account data
let bump = data[BUMP_OFFSET];
```

Apply the same pattern to Record and derived PDAs (e.g., Mint, Group).

Remediation

Marked as won't fix.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I5 Non-Specific Error Codes Hinder Debugging

INFO | WONTFIX

Description

We found that the program overwhelmingly uses ProgramError::InvalidAccountData for a wide variety of error conditions, making debugging and error diagnosis extremely difficult. The vast majority of Error codes use InvalidAccountData regardless of the actual failure reason.

Different failure conditions that all return the same error:

- Frozen state violations
- Incorrect discriminators
- Mismatched pubkeys
- Class mismatch
- Owner type validation failures
- Permission check failures
- Token supply validation
- Metadata validation failures

This makes it nearly impossible to diagnose which specific validation failed when a transaction errors out. Developers, auditors, and users have no way to distinguish between fundamentally different error conditions.

Location

Widespread throughout the codebase:

- state/record.rs
- state/class.rs
- instructions/mint_tokenized_record.rs
- instructions/update_record.rs

Relevant Code

```
// state/record.rs - Many different failures, same error

// Line 74 - Discriminator check
if data[DISCRIMINATOR_OFFSET].ne(&Self::DISCRIMINATOR) {
    return Err(ProgramError::InvalidAccountData);
}

// Line 90 - Permission check
if class_data[IS_PERMISSIONED_OFFSET].ne(&1u8) {
    return Err(ProgramError::InvalidAccountData);
}

// Line 116 - Mint validation
if mint.key().ne(&data[OWNER_OFFSET..OWNER_OFFSET + size_of::<Pubkey>()]) {
    return Err(ProgramError::InvalidAccountData);
}

// Line 165 - Class mismatch
if class.key().ne(&data[CLASS_OFFSET..CLASS_OFFSET + size_of::<Pubkey>()]) {
    return Err(ProgramError::InvalidAccountData);
}

// Line 197 - Owner type check
if data[OWNER_TYPE_OFFSET].ne(&(OwnerType::Pubkey as u8)) {
    return Err(ProgramError::InvalidAccountData);
}
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
// Line 320 - Frozen state
if data[IS_FROZEN_OFFSET].eq(&1u8) {
    return Err(ProgramError::InvalidAccountData);
}
```

Mitigation Suggestion

Define custom error enum with specific error codes for different failure conditions:

```
// Add to a new errors.rs file or in lib.rs
use solana_program::{
    decode_error::DecodeError,
    program_error::ProgramError,
};
use thiserror::Error;

#[derive(Clone, Debug, Eq, Error, PartialEq)]
pub enum RecordServiceError {
    #[error("Account is frozen")]
    FrozenAccount,

    #[error("Invalid discriminator")]
    InvalidDiscriminator,

    #[error("Class mismatch")]
    ClassMismatch,

    #[error("Owner mismatch")]
    OwnerMismatch,

    #[error("Invalid owner type")]
    InvalidOwnerType,

    #[error("Permission denied")]
    PermissionDenied,

    #[error("Class not permissioned")]
    ClassNotPermissioned,

    #[error("Invalid token supply")]
    InvalidTokenSupply,

    #[error("Mint mismatch")]
    MintMismatch,

    #[error("Invalid metadata length")]
    InvalidMetadataLength,

    #[error("Name too long")]
    NameTooLong,

    #[error("Account already initialized")]
    AccountAlreadyInitialized,
}

impl From<RecordServiceError> for ProgramError {
    fn from(e: RecordServiceError) -> Self {
        ProgramError::Custom(e as u32)
    }
}

impl<T> DecodeError<T> for RecordServiceError {
    fn type_of() -> &'static str {
        "RecordServiceError"
    }
}
```

Then use specific errors throughout:

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
// Instead of:
if data[IS_FROZEN_OFFSET].eq(&1u8) {
    return Err(ProgramError::InvalidAccountData);
}

// Use:
if data[IS_FROZEN_OFFSET].eq(&1u8) {
    return Err(RecordServiceError::FrozenAccount.into());
}

// Instead of:
if data[DISCRIMINATOR_OFFSET].ne(&Self::DISCRIMINATOR) {
    return Err(ProgramError::InvalidAccountData);
}

// Use:
if data[DISCRIMINATOR_OFFSET].ne(&Self::DISCRIMINATOR) {
    return Err(RecordServiceError::InvalidDiscriminator.into());
}
```

Remediation

Marked as won't fix.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I6 Arbitrary Debug Logging Statements Waste Compute Units

INFO FIXED

Description

We found multiple `sol_log_64(0, 0, 0, 0, 0)` debug logging statements are scattered throughout the codebase, particularly in the `CreateRecord` instruction. These appear to be leftover debugging artifacts from development that serve no purpose in production code. Each logging call consumes compute units unnecessarily, increasing transaction costs for users.

Location

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/create_record.rs#L51

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/create_record.rs#L56

https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/instructions/create_record.rs#L84

Relevant Code

```
// create_record.rs line 51
impl<'info> TryFrom<&'info [AccountInfo]> for CreateRecordAccounts<'info> {
    type Error = ProgramError;

    fn try_from(accounts: &'info [AccountInfo]) -> Result<Self, Self::Error> {
        let [owner, payer, class, record, _system_program, rest @ ..] = accounts else {
            return Err(ProgramError::NotEnoughAccountKeys);
        };

        sol_log_64(0, 0, 0, 0, 0);  // Line 51 - No functional purpose

        // Check class permission
        Class::check_permission(class, rest.first())?;

        sol_log_64(0, 0, 0, 0, 0);  // Line 56 - No functional purpose
        // ...
    }
}

// create_record.rs line 84
impl<'info> TryFrom<Context<'info>> for CreateRecord<'info> {
    type Error = ProgramError;

    fn try_from(ctx: Context<'info>) -> Result<Self, Self::Error> {
        sol_log_64(0, 0, 0, 0, 0);  // Line 84 - No functional purpose
        // ...
    }
}
```

Mitigation Suggestion

Remove all debug logging statements that serve no functional purpose:

Remediation

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

Fixed in
https://github.com/solana-foundation/solana-record-service/commit/5293929c3db15adeac06e1355d6c3c3cbdd3ad3b

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I7 Discriminators Not Defined in Centralized Location

INFO WONTFIX

Description

Account discriminators are scattered throughout the codebase rather than being defined in a single, centralized location. This creates multiple issues:

• Makes it difficult to ensure discriminators don't collide

• Harder to audit all discriminator values at once

• Increases risk of accidental reuse of discriminator values

• Makes maintenance more difficult when adding new account types

Currently, discriminators are defined in:

• `Class::DISCRIMINATOR = 1` in `state/class.rs`

• `Record::DISCRIMINATOR = 2` in `state/record.rs`

• `CLOSED_ACCOUNT_DISCRIMINATOR = 0xff` in `constants.rs`

Location

• https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/state/class.rs#L26

• https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/state/record.rs#L50

• https://github.com/solana-foundation/solana-record-service/blob/ba0f64964eccc0b559ee01012d0d547aa52cec00/program/src/constants.rs#L4

Relevant Code

```
// state/class.rs line 26
impl<'info> Class<'info> {
    pub const DISCRIMINATOR: u8 = 1;
    // ...
}

// state/record.rs line 50
impl<'info> Record<'info> {
    pub const DISCRIMINATOR: u8 = 2;
    // ...
}

// constants.rs line 4
pub const CLOSED_ACCOUNT_DISCRIMINATOR: u8 = 0xff;
```

Mitigation Suggestion

Centralize all discriminator definitions in a single location for better maintainability and collision prevention:

```
// constants.rs or new discriminators.rs file
/// Account discriminators - centralized to prevent collisions
pub mod discriminators {
    /// Uninitialized account (default state)
    pub const UNINITIALIZED: u8 = 0x00;

    /// Class account discriminator
    pub const CLASS: u8 = 0x01;
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    /// Record account discriminator
    pub const RECORD: u8 = 0x02;

    /// Closed/deleted account discriminator
    pub const CLOSED_ACCOUNT: u8 = 0xff;
}

// Then use throughout codebase:
impl<'info> Class<'info> {
    pub const DISCRIMINATOR: u8 = discriminators::CLASS;
    // ...
}

impl<'info> Record<'info> {
    pub const DISCRIMINATOR: u8 = discriminators::RECORD;
    // ...
}
```

Remediation

Issue has been marked as won't fix.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:
• Solana-specific vulnerabilities
• Access control issues
• Arithmetic errors and precision loss
• Race conditions and MEV opportunities
• Logic errors and edge cases
• Performance optimization opportunities
• Invariant violations
• Account confusion vulnerabilities
• Authority check omissions
• Token22 implementation risks and SPL-related pitfalls
• Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.